12.ROS2 参数说明

1.参数简介

类似 C++编程中的全局变量，可以便于在多个程序中共享某些数据，参数是 ROS 机器人系统中的全局字典，可以运行多个节点中共享数据。

在 ROS 系统中，参数是以全局字典的形态存在的，什么叫字典？就像真实的字典一样，由名称和数值组成，也叫做键和值，合成键值。或者我们也可以理解为，就像编程中的参数一样，有一个参数名，然后跟一个等号，后边就是参数值了，在使用的时候，访问这个参数名即可。

参数的特性非常丰富，比如某一个节点共享了一个参数，其他节点都可以访问，如果某一个节点对参数进行了修改，其他节点也有办法立刻知道，从而获取最新的数值。

2.param 参数

我们先来了解下 param 参数指令，详细说明如下：

指令	说明
ros2 param list	列出当前多个参数
ros2 get param_key	显示某个参数值
ros2 set param_key param_value	设置某个参数值
ros2 param dump file_name	保存参数到文件
ros2 param load file_name	从文件读取参数
ros2 param delete param_key	删除参数

3.创建参数案例

1） 使用 Nomachine 远程连接工具连接 Jetson Orin Nano 主板，然后双击桌面的 

，输入指令“**cd ros2_ws/src/**”指令，切换到 ros2_ws 工作空间的 src 文件夹中。

```
ubuntu@ubuntu:~$ cd ros2_ws/src/
ubuntu@ubuntu:~/ros2_ws/src$
```

2） 输入“**ros2 pkg create param_demo --build-type ament_python --dependencies rclpy**”并按下回车，创建一个名为“**param_demo**”的功能包，添加依赖关系 rclpy。

```
ubuntu@ubuntu:~/ros2_ws/src$ ros2 pkg create param_demo --build-type ament_python --dependencies rclpy
going to create a new package
package name: param_demo
destination directory: /home/ubuntu/ros2_ws/src
package format: 3
```

3） 输入“**cd param_demo/param_demo/**”指令，切换到 param_demo 功能包下。

```
ubuntu@ubuntu:~/ros2_ws/src$ cd param_demo/param_demo/
ubuntu@ubuntu:~/ros2_ws/src/param_demo/param_demo$
```

1） 输入指令“**gedit param_demo.py**”编辑程序,复制下面程序,使用快捷键“ctrl+s”保存并退出。

```
ubuntu@ubuntu:~/ros2_ws/src/param_demo/param_demo$ gedit param_demo.py
```

```
import rclpy  # 导入 rclpy 模块

from rclpy.node import Node  # 导入 Node 类

from rclpy.parameter import Parameter  # 导入 Parameter 类



class MinimalParam(Node):  # 定义一个继承自 Node 类的 MinimalParam 类

    def __init__(self):
```

```
        super().__init__('minimal_param_node')  # 调用父类构造函数初始化节点

        self.declare_parameter('my_parameter', 'ros2')  # 声明一个名为'my_parameter'的参数，并设置默认值为'ros2'

        self.timer = self.create_timer(1, self.timer_callback)  # 创建定时器，设置回调函数为 timer_callback，时间间隔为 1 秒

    def timer_callback(self):
        my_param = self.get_parameter('my_parameter').get_parameter_value().string_value  # 获取参数'my_parameter'的值，并转换为字符串

        self.get_logger().info('Hello %s!' % my_param)  # 打印带有参数值的日志消息

        my_new_param = Parameter(  # 创建一个新的参数对象
            'my_parameter',  # 参数名称为'my_parameter'
            rclpy.Parameter.Type.STRING,  # 参数类型为字符串
            'ros2'  # 参数值为'ros2'
        )
        all_new_parameters = [my_new_param]  # 将新的参数对象放入列表中
        self.set_parameters(all_new_parameters)  # 设置节点的参数值为新的参
```

```
数值



def main():

    rclpy.init()  # 初始化 ROS 节点

    node = MinimalParam()  # 创建 MinimalParam 对象

    rclpy.spin(node)  # 进入主循环



if __name__ == '__main__':

    main()
```

2） 输入指令“**chmod +x param_demo.py**”，并按下回车，为保存的 param_demo.py 赋予可执行权限。

```
ubuntu@ubuntu:~/ros2_ws/src/param_demo/param_demo$ chmod +x param_demo.py
```

4.setup.py 文件设置

setup.py 文件是定义一个 ROS2 包的远数据和构建配置，提供了包的元数据、依赖关系、构建配置和安装逻辑等信息，可以帮助开发者正确构建、安装和使用 ROS2 包。需要将 param_demo.py 的程序入口写到 setup.py 文件中。

1） 输入“**cd ..**”指令，切换到上一级目录。

```
ubuntu@ubuntu:~/ros2_ws/src/param_demo/param_demo$ cd ..
ubuntu@ubuntu:~/ros2_ws/src/param_demo$
```

2） 输入“**gedit setup.py**”指令，并按下回车，打开 setup.py 文件。

```
ubuntu@ubuntu:~/ros2_ws/src/action_demo$ gedit setup.py
```

复制下面程序,使用快捷键“ctrl+s”保存并退出。

```
'param_demo = param_demo.param_demo:main'\
```

```
17    maintainer_email='ubuntu@todo.todo',
18    description='TODO: Package description',
19    license='TODO: License declaration',
20    tests_require=['pytest'],
21    entry_points={
22        'console_scripts': [
23            'param_demo = param_demo:param_demo:main'
24        ],
25    },
26 )
```

4.编译运行

3） 使用 Nomachine 远程连接工具连接 Jetson Orin Nano 主板，然后双击桌面的 

，输入指令“**cd ~/ros2_ws/**”指令，切换到工作空间的目录。

```
ubuntu@ubuntu:~$ cd ~/ros2_ws/
ubuntu@ubuntu:~/ros2_ws$
```

4） 输入“**colcon build**”指令，并按下回车，编译工作空间中的功能包。

```
ubuntu@ubuntu:~/ros2_ws$ colcon build
```

5） 再输入指令“**source ./install/setup.bash**”，并按下回车，让环境变量生效。

```
ubuntu@ubuntu:~/ros2_ws$ source ./install/setup.bash
```

6） 输入指令“**ros2 run param_demo param_demo**”，并按下回车，启动 param_demo 节点。

```
ubuntu@ubuntu:~/ros2_ws$ ros2 run param_demo param_demo
[INFO] [1709031793.580104006] [minimal_param_node]: Hello ros2!
[INFO] [1709031794.564565924] [minimal_param_node]: Hello ros2!
[INFO] [1709031795.564572765] [minimal_param_node]: Hello ros2!
```

7） 使用 Nomachine 远程连接工具连接 Jetson Orin Nano 主板，然后双击桌面的 

，参考步骤 1）-3）“**ros2 param set minimal_param_node my_parameter world**”，并按下回车，修改 minimal_param_node 这个节点的 my_parameter 参数为 world。

```
ubuntu@ubuntu:~$ ros2 param set minimal_param_node my_parameter world
Set parameter successful
```

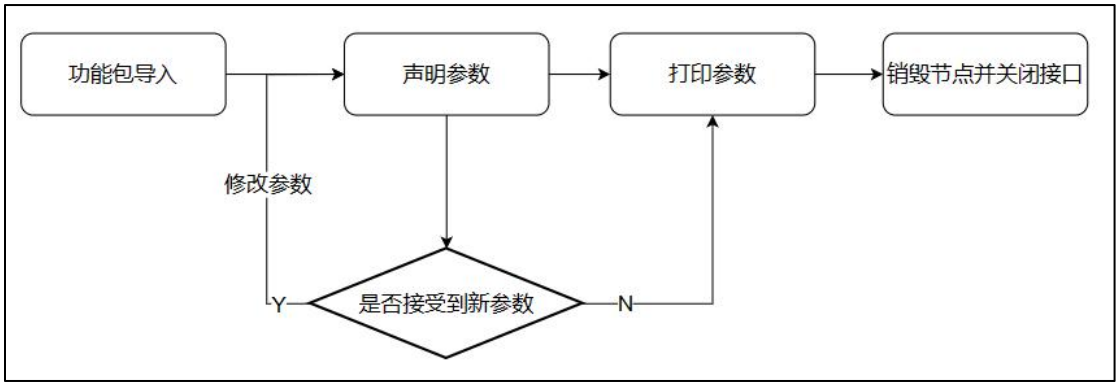
这时候可以发现 param_demo 节点输出的信息为“**Hello world!**”，说明参数修改成

功。

```
[INFO] [1709031867.564597356] [minimal_param_node]: Hello world!
```

4.程序分析

根据实现的效果，梳理程序的过程逻辑，如下图所示：



创建了一个 MinimalParam 类,在构造函数中声明了一个字符串参数"my_parameter",并创建了一个 1 秒间隔的定时器。定时器回调函数每隔 1 秒会被调用,在回调内它首先获取参数当前的值,然后打印日志,再创建一个新的参数对象并设置新值,最后调用 set_parameters 函数来修改参数服务器中的参数值。主函数初始化 ROS 节点和环境,创建节点实例,并进入主循环来驱动定时器回调的执行。

- 主函数

```
def main():
    rclpy.init()  # 初始化ROS节点
    node = MinimalParam()  # 创建MinimalParam对象
    rclpy.spin(node)  # 进入主循环
```

首先调用 rclpy.init()对 ROS2 Python 接口进行初始化，然后实例化 MinimalParam，然后执行 node.run()函数。

- MinimalParam 类

```
class MinimalParam(Node):  # 定义一个继承自Node类的MinimalParam类
    def __init__(self):
        super().__init__('minimal_param_node')  # 调用父类构造函数初始化节点

        self.declare_parameter('my_parameter', 'hiwonder')  # 声明一个名为'my_parameter'的参数，并设置默认值为'hiwonder'

        self.timer = self.create_timer(1, self.timer_callback)  # 创建定时器，设置回调函数为timer_callback，时间间隔为1秒

    def timer_callback(self):
        my_param = self.get_parameter('my_parameter').get_parameter_value().string_value  # 获取参数'my_parameter'的值，并
转换为字符串

        self.get_logger().info('Hello %s!' % my_param)  # 打印带有参数值的日志消息

        my_new_param = Parameter(  # 创建一个新的参数对象
            'my_parameter',  # 参数名称为'my_parameter'
            rclpy.Parameter.Type.STRING,  # 参数类型为字符串
            'hiwonder'  # 参数值为'hiwonder'
        )
        all_new_parameters = [my_new_param]  # 将新的参数对象放入列表中
        self.set_parameters(all_new_parameters)  # 设置节点的参数值为新的参数值
```

首先声明了一个名为"my_parameter"的字符串参数,并创建了一个 1 秒周期的定时器. 定时器回调函数 timer_callback 会每秒被调用。它首先获取"my_parameter"参数目前的值,然后打印日志输出。紧接着,回调函数创建一个新参数对象,设置相同名称但值为"ros2"的新参数值。之后调用 set_parameters 函数来修改参数值。。